STUDENT GRADE MANAGEMENT SYSTEM

Full-Stack Programming Project & Software Process Measurement

Course / Project	Software Quality and Improvement
Lecturer	Dr. Tesfaye Biru
Student name	Matan Workneh
ID number	0143
Section	2
Programming languages	Elixir (backend), TypeScript (frontend)
Application type	Web application: Phoenix JSON API and Next.js interface
Source code	https://github.com/matan-workneh7/student_grading_webapp

Project objective

Develop a full-stack Student Grade Management System. The system accepts students, subjects and marks, validates every input, calculates the total, weighted average and letter grade, and presents class statistics, student reports and exports. The project also applies process measurement by recording estimated and actual time for each job.

1. Project Overview

The Student Grade Management System is a full-stack application. The backend is an Elixir Phoenix API that stores data in SQLite. The frontend is a Next.js application that talks to the API, and it can also run on its own with built-in mock data. Users record students, subjects and marks, and the system calculates totals, weighted averages and letter grades.

1.1 Scope

Included: students, subjects with credit hours, marks from 0 to 100 inclusive, total and weighted average, letter grade from a configurable scale, a stateless grade calculator, student reports with class rank, class statistics, searchable and paginated student list, CSV export, and a mock mode for the interface.

Excluded: user accounts and authentication, deployment configuration, multiple grading policies at the same time, and report formats other than CSV.

1.2 Assumptions

Each mark belongs to one student and one subject. A subject's credit hours are its weight in the average (default 1), so equal weights give the plain arithmetic mean. A student needs at least one mark to receive a result. The grading scale is the one proposed for this assignment and is not an institutional policy.

2. Requirements Analysis (Job 1)

The requirements below describe what the system must do and the constraints it must satisfy before implementation.

ID	Requirement	Category
FR1	Create, edit and delete students (name required; student number and section optional).	Functional
FR2	Create, edit and delete subjects with a unique code and credit hours greater than 0 (default 1).	Functional
FR3	Record a mark for each student and subject; accept only numeric marks from 0 through 100 inclusive.	Functional
FR4	Reject empty names, non-numeric marks, out-of-range marks and invalid weights.	Functional
FR5	Calculate the total and the weighted average of a student's marks.	Functional
FR6	Assign the letter grade from the unrounded average using the grading scale.	Functional
FR7	Display averages rounded to two decimal places.	Functional
FR8	Let the user edit the grading scale, and validate it (descending, last minimum 0).	Functional
FR9	Provide a stateless calculator that returns the result without storing anything, with an option to save the result as a student.	Functional
FR10	List students with search, sorting and pagination.	Functional
FR11	Show a student report with total, weighted average, letter grade and class rank.	Functional
FR12	Show class statistics: student count, class average, highest, lowest, grade distribution and per-subject averages.	Functional
FR13	Export students and results as CSV.	Functional
FR14	Provide a dashboard that presents the class statistics.	Functional
NFR1	Show clear validation messages, including server field errors on the matching form fields, and recover without losing input.	Non-functional
NFR2	Allow the interface to run without the backend (mock mode) and switch to the real API by configuration.	Non-functional
NFR3	Provide a responsive layout with light and dark themes.	Non-functional
NFR4	Keep frontend and backend independent, connected only by a documented JSON API.	Non-functional

2.1 Grading scale

Average mark	Letter grade
90 - 100	A
80 - 89.99	B
70 - 79.99	C
60 - 69.99	D
50 - 59.99	E
Below 50	F

The unrounded average decides the letter grade. The displayed average is rounded to two decimal places. The scale is stored in the database and can be changed in the settings page.

3. Design (Job 2)

The design separates grading rules, data access, the HTTP layer and the user interface so that each part can be tested on its own.

3.1 Architecture

The browser runs the Next.js application (port 3000). It sends JSON requests to the Phoenix API (port 4000) under /api/v1. The API reads and writes SQLite through Ecto. When mock mode is on, the interface uses an in-memory client that implements the same interface and the same response shapes, so the frontend can be developed and demonstrated without the backend.

3.2 Backend components

Module	Responsibility
GradeApi.Grading	Pure grading rules: mark validation, weighted average, letter lookup, result summary. No database access.
GradeApi.Students, Subjects, Grades	Create, read, update and delete records; paging and search for students; saving marks.
GradeApi.Scale and GradingScale schema	Reads and stores the grading scale and validates it.
GradeApi.Reports	Student report, class statistics, ranking and CSV export.
GradeApiWeb.ApiController	One action per route; builds the JSON responses.
GradeApiWeb.FallbackController, ErrorJSON	Render the 404 and 422 error envelopes.
GradeApiWeb.Cors	Allows the frontend origin and answers preflight requests.

3.3 Frontend components

Part	Responsibility
lib/api (index.ts, grading.ts)	Typed API client with zod schemas for every response, the mock client, and the pure grading helper.
lib/env.ts	Reads NEXT_PUBLIC_API_URL and NEXT_PUBLIC_USE_MOCKS and validates them.
hooks/use-api.ts	TanStack Query hooks for fetching and mutations.
app/ pages	Dashboard, calculator, students, student report, subjects and settings.
components/	Application shell, providers and shadcn/ui building blocks.

3.4 Data model

Table	Main fields
students	id, name, student_number (unique, optional), section (optional), inserted_at
subjects	id, name, code (unique), credit_hours (greater than 0, default 1)

Table	Main fields
grades	student_id, subject_id, mark (0 to 100); one mark per student and subject
grading scale	ordered list of {min, letter} rows, descending by min, ending at min 0

3.5 API routes (base URL /api/v1)

Method and path	Purpose
GET /health	Service status
GET, POST /students	List (search, sort, paging) and create students
GET, PATCH, DELETE /students/:id	Read, update and delete a student
PUT /students/:id/grades	Save marks for a student
DELETE /students/:id/grades/:subject_id	Remove one mark
GET /students/:id/report	Total, weighted average, letter grade and rank
GET, POST /subjects; PATCH, DELETE /subjects/:id	Manage subjects
POST /calculate	Stateless calculation, nothing is stored
GET, PUT /grading-scale	Read and replace the grading scale
GET /stats	Class statistics for the dashboard
GET /export/students.csv	CSV export

Lists return {data, meta:{page, per_page, total}} and single records return {data}. A validation failure returns HTTP 422 with {error:{code:"validation_failed", message, fields}}, and a missing record returns HTTP 404 with {error:{code:"not_found", message}}.

3.6 Process sequence

Step	Process
1	The user enters a student and subject marks in a form.
2	The form validates the entries in the browser and shows errors next to the fields.
3	The client sends the request to the API (or to the mock client in mock mode).
4	The API validates again and rejects invalid input with a 422 response.
5	The grading module calculates the total, weighted average and letter grade from the unrounded average.
6	The interface displays the result, the report or the updated statistics.

4. Implementation (Job 3)

The code is in the GitHub repository given on the cover page. This section lists the technology, the size of the work and where each requirement is implemented.

4.1 Technology and versions

Component	Version
Elixir / Phoenix / Ecto / ecto_sqlite3 / Bandit	Elixir ~> 1.17; Phoenix 1.8.16; Ecto 3.14.3; ecto_sqlite3 0.26.0; Bandit 1.14.1
Next.js / React / TypeScript	16.4.3 / 19.3.1 / 5.9.3
Tailwind CSS / shadcn components	4.3.4 / Base UI style with Phosphor icons
TanStack Query / react-hook-form / zod	5.106.1 / 7.91.0 / 4.6.7
Recharts / Vitest / ESLint	3.8.0 / 5.1.0 / 10.14.0

4.2 Size of the work

Folder	Lines
backend/lib (21 modules, 73 public functions, 4 schemas, 18 routes)	1,028

Folder	Lines
backend/test	240
backend/priv (1 migration, seeds)	101
next-app/app (6 pages and the layout)	432
next-app/components (excluding generated shadcn/ui)	115
next-app/hooks	37
next-app/lib	210
Total	2,163

The counts exclude the shadcn/ui component files (2,655 lines), which were added with the shadcn tool.

4.3 Where each requirement is implemented

Requirement	Location
Mark 0 to 100 inclusive	backend/lib/grade_api/grading.ex (validate_mark/1); calculator form schema in next-app/app/calculator/page.tsx
Weighted average and total	backend/lib/grade_api/grading.ex (weighted_average/1, summarize/2); next-app/lib/api/grading.ts
Letter from unrounded average	grading.ex (letter_for/2 receives average_raw); grading.ts (gradeLetter)
Scale validation	backend/lib/grade_api/schemas/grading_scale.ex (valid_scale?/1); scaleSchema in next-app/lib/api/index.ts
Pagination and search	backend/lib/grade_api/students.ex; next-app/app/students/page.tsx
Statistics and ranking	backend/lib/grade_api/reports.ex; next-app/app/page.tsx
CSV export	backend/lib/grade_api/reports.ex; export button in the students page
422 and 404 shapes	backend/lib/grade_api_web/controllers (api_controller.ex, fallback_controller.ex, error_json.ex)
CORS	backend/lib/grade_api_web/cors.ex
Mock mode switch	next-app/lib/env.ts; client selection in next-app/lib/api/index.ts

5. Testing (Job 4)

Automated tests were run and their results observed on 6 November 2026. The commands and results are listed in 5.2.

5.1 Test cases and outcomes

Test	Condition	Where it is checked	Outcome
T1	Marks 80, 90, 70: total 240, average 80.00, grade B	Backend test "basic average and thresholds"; frontend test "calculates 80/90/70 as 240 / 80.00 / B"; live API call	Passed
T2	One subject with mark 100: grade A	Backend grading test	Passed
T3	Average exactly 90: grade A	Backend and frontend boundary tests	Passed
T4	Average exactly 80: grade B	Backend and frontend boundary tests	Passed
T5	Average exactly 50: grade E	Backend and frontend boundary tests	Passed
T6	Mark -1 or 101 rejected	Backend test "mark validation rejects invalid values"; frontend test; live API call returned HTTP 422	Passed
T7	Non-numeric mark (abc) rejected	Backend mark validation test	Passed
T8	Invalid subject count (zero, negative, non-integer)	The interface adds and removes rows instead of asking for a count; the API rejects an empty subject list. No automated test	Not tested automatically
T9	Empty student or subject name rejected	Form schema and student changeset implement the rule. No automated test	Not tested automatically

Test	Condition	Where it is checked	Outcome
T10	Process another student	The calculator can be filled in again and a result saved as a student. No automated test	Not tested automatically
T11	Weighted average (90 with weight 3, 60 with weight 1)	Backend and frontend weighted tests	Passed
T12	Unrounded average 89.996 gets B while 90.00 is displayed	Backend test "weights and unrounded grade boundaries are preserved"	Passed
T13	404 and 422 responses, calculator route, grades, report, scale, statistics, CSV	Backend controller tests	Passed

5.2 Verification results

Check	Command	Result
Backend tests	mix test (in backend)	9 passed, 0 failed
Backend formatting	mix format --check-formatted	Clean
Frontend tests	bun run test (Vitest)	4 passed, 0 failed
Lint	bun run lint (ESLint)	No errors
Type check	bun run check-types (tsc --noEmit)	No errors
Production build	bun run build	Compiled; 8 pages generated
Live API	GET /health	{"status":"ok"}
Live API	POST /calculate with 80, 90, 70	total 240, average 80.0, letter B
Live API	POST /calculate with mark 101	HTTP 422, validation_failed, "must be a number between 0 and 100"

In total 13 automated test cases ran (9 backend, 4 frontend) and all passed. The interface was also checked by hand in mock mode, and the six pages were opened against the real API.

6. Documentation (Job 5)

The repository contains a README with a project description, screenshots of the overview, students, subjects, grade calculator and settings pages, the grading rules, the technology, the project structure, and run and test instructions. This report is the second document.

6.1 How to run

1. Install Elixir with Erlang/OTP, and Bun.
2. Backend: `cd backend`, then `mix setup`, then `mix phx.server` (port 4000).
3. Frontend with mock data: `cd next-app`, `bun install`, then `NEXT_PUBLIC_USE_MOCKS=true bun run dev`.
4. Frontend with the real API: `NEXT_PUBLIC_USE_MOCKS=false NEXT_PUBLIC_API_URL=http://localhost:4000/api/v1 bun run dev`.
5. Open http://localhost:3000. Tests: `mix test` in backend and `bun run test` in next-app.

6.2 Source code

The complete source code is available at https://github.com/matan-workneh7/student_grading_webapp.

7. Software Process Measurement

An estimate is a prediction made before a job. Actual time is the time the job took. A work-product unit is a countable output of a job. Units are defined separately for each job and are not added across jobs.
Note on the records. The actual times are the totals the author recorded for each job. The estimates were not written down before the jobs started, so they are retrospective: they are the effort a junior software engineer would reasonably predict for each job on a project of this size. The dates are reconstructed from file

timestamps and the date of the first commit, not read from a stopwatch.

7.1 Job Number Log

Job	Work-product unit	Est. (retrospective)	Actual time	Actual units
1. Requirements	Requirements documented	25 min	31 min	18 requirements (14 functional, 4 non-functional)
2. Design	Design artifacts	60 min	87 min	3 artifacts (architecture, data model, API contract)
3. Implementation	Code produced	150 min	174 min	73 public Elixir functions, 8 React hooks, 6 pages
4. Testing	Test cases executed	45 min	38 min	13 test cases
5. Documentation	Documents prepared	30 min	24 min	2 documents (README and this report)
Total time		310 min	354 min	Units are not additive

7.2 Daily Time Log

Date	Job	Activity	Est. min	Actual min
05-Nov-2026	1	Define scope and write requirements	25	31
05-Nov-2026	2	Design the architecture, data model and API contract	60	87
05-Nov-2026	3	Implement the frontend and the Phoenix backend	150	174
05 and 06-Nov-2026	4	Run test cases and correct defects	45	38
06-Nov-2026	5	Prepare the README and this report	30	24
		Total	310	354

7.3 Estimate against actual

Job	Difference (actual - estimate)	Percent
1. Requirements	+6 min	+24%
2. Design	+27 min	+45%
3. Implementation	+24 min	+16%
4. Testing	-7 min	-16%
5. Documentation	-6 min	-20%
Total	+44 min	+14%

Design and implementation took longer than predicted. The design had to include an API contract that both the frontend and backend would follow, and the implementation included correcting the frontend after it was first built against an incorrect assumed contract. Testing and documentation were shorter than predicted because the grading logic was small and well-defined.

8. Defects, Limitations and Possible Improvements

8.1 Defects found and fixed

Where	Defect	Status
next-app/components/theme-provider.tsx	Keyboard handler read event.key.toLowerCase() and crashed when the browser sent a key event without a key (for example on autofill).	Found in the browser console; fix applied (safe access to event.key)
Theme toggle button	Server and client rendered different icons, which caused a React hydration mismatch.	Found in the browser console; fix applied (icon no longer depends on the theme during render)

Where	Defect	Status
next-app/lib/api	The API client was first written against an assumed contract and did not match the real endpoints and error shapes.	Fixed: client and schemas rewritten to the documented contract; checked by type-check, tests and build

8.2 Defects found by code review and not fixed

A read-only review of the source found the following. They are reported here so the results are not overstated.

Where	Finding
backend/lib/grade_api/schemas/grading_scale.ex	The backend accepts duplicate thresholds and minimums outside 0 to 100; the frontend schema rejects them, so some scales could be stored but fail to load in the interface.
backend/lib/grade_api_web/controllers/api_controller.ex	The calculate action checks marks but not subject names or weights.
backend/lib/grade_api/grades.ex	Marks are saved one by one without a database transaction, so a later invalid entry could leave earlier ones saved.
backend FallbackController and Students.result/1	Defined but not referenced by the controller; unused code.
backend error_json.ex	The generic server-error response uses a different envelope from the 404 and 422 responses.
next-app/lib/api/index.ts (mock client)	The mock client differs from the backend in CSV columns, search fields, default sort and the per-page limit, and its statistics fail for a student with no marks.
next-app/app/calculator/page.tsx	A blank mark is accepted as 0. Saving a result creates the student, subjects and marks in separate requests that are not atomic.
next-app/app/students/page.tsx	Each row requests its own report to show the average and grade.
Tests	No automated tests cover empty names, blank marks, scale validation parity, mock parity or partial writes.

8.3 Limitations

- There is no authentication or authorization.
- The allowed CORS origin is fixed to http://localhost:3000.
- One grading scale applies to the whole system.
- Data is stored in a local SQLite file and there is no deployment configuration.
- The mock client keeps data in memory only.
- The calculator page has no reset button.

8.4 Possible improvements

- Fix the defects in 8.2, starting with matching scale validation and transactional saves.
- Return averages and letters in the student list response.
- Add authentication with separate lecturer and student roles.
- Add an endpoint that saves a calculator result atomically.
- Add contract tests that run the same checks against the mock client and the real API.
- Add per-class and per-term reports and support for several grading policies.

9. Conclusion

The project delivers a full-stack Student Grade Management System. Its grading rules are marks from 0 to 100, a weighted average, and a letter grade decided from the unrounded average. The grading logic is isolated in a pure module and covered by automated tests in both the backend and the frontend, and 13 automated test cases passed. A review of the code found defects that are reported openly in this document.

The project also applied process measurement: five jobs with estimated and actual time, work-product units and a comparison showing the actual effort was 14% above the retrospective estimate.